



ABSTRAÇÃO, TIPOS E ESTRUTURAS DE DADOS

Arrays: Organizando Coleções de Dados

Depois de definir tipos e as ações que podemos realizar sobre eles, chegou a hora de organizar coleções inteiras desses dados.



O que é e por que usar um Array?

Conceito

Uma estrutura que armazena vários dados do mesmo tipo em uma única variável.

Por que usar?

- Organiza informações de forma eficiente, evitando criar muitas variáveis individuais.
- Facilita o acesso, a busca e a modificação de dados.

Estrutura

- Índices: posições numeradas que identificam cada elemento.
- Elementos: os valores armazenados em cada posição.



Exemplo cotidiano

Uma prateleira com várias caixas numeradas, cada uma guardando um item.

EXEMPLO DE CÓDIGO

```
nomes = ["Ana", "João", "Maria", "Lucas"]
```

```
nomes[0] → "Ana"
```

```
nomes[2] → "Maria"
```



Dimensões de um Array



1D — Unidimensional

Armazena uma sequência linear de elementos.

Exemplo: Lista com os nomes dos alunos de uma turma.

```
alunos = ["Ana", "João",  
          "Maria", "Lucas"]  
print(alunos[2])  
# Exibe: Maria
```

Uso cotidiano

Lista de contatos do celular ou lista de músicas em um app.



2D — Bidimensional

Armazena dados em linhas e colunas, como uma planilha.

Exemplo: Notas dos alunos em diferentes provas.

```
notas = [[8.5, 9.0, 7.5],  
         [7.0, 8.0, 6.5],  
         [9.5, 8.5, 10.0]]  
print(notas[1][2])  
# Exibe: 6.5
```

Uso cotidiano

Tabela de horários de ônibus ou boletim escolar.



Multidimensional

Armazena dados em mais de duas dimensões.

Exemplo: Representação de uma imagem (cada pixel tem R, G, B).

```
imagem = [[[255, 0, 0],  
           [0, 255, 0]], [[0, 0, 255],  
                          [255, 255, 0]]]  
print(imagem[0][1])  
# Exibe: [0, 255, 0]
```

Uso cotidiano

Fotos digitais, mapas, planilhas 3D ou jogos (x, y, z).



Aninhamento: Arrays de Arrays

Um array aninhado é um conjunto dentro de outro conjunto — uma lista que contém outras listas. É uma forma de organizar informações em camadas, como caixas dentro de caixas.



Notas dos alunos

```
Ana → [8.5, 9.0, 7.5]
João → [7.0, 6.5, 8.0]
Maria → [10.0, 9.5, 9.0]
```

O conjunto de listas forma um array bidimensional (array de arrays) que representa uma turma inteira.



Cardápio do restaurante

```
Pratos → [Lasanha, Feijoadada, Peixe]
Sobremesas → [Pudim, Sorvete, Bolo]
Bebidas → [Suco, Refri, Café]
```

O cardápio completo é um array de arrays: guarda várias listas dentro de uma só.



Mala de viagem

```
Saco 1 → [camisas, calças]
Saco 2 → [meias, sapatos]
Saco 3 → [produtos de higiene]
```

A mala é um array e cada saco é outro array: juntos formam um array aninhado.



Arrays como Contêineres Universais



Flexibilidade total

Um array pode ser visto como um contêiner universal porque guarda diferentes elementos organizados em sequência. Possui estrutura consistente — funciona do mesmo jeito independentemente do conteúdo (números, textos, imagens) — mantendo o mesmo formato, índice e operações básicas: acessar, adicionar, remover, percorrer. Essa consistência torna os arrays previsíveis e fáceis de usar, permitindo aplicar os mesmos algoritmos a diferentes tipos de dados.



Poder computacional

Os arrays concentram grande poder computacional porque permitem armazenar e processar grandes quantidades de dados de forma organizada e eficiente. Sua estrutura uniforme possibilita operações rápidas e repetitivas — busca, ordenação, análise em massa — essencial para o pensamento computacional, pois favorece algoritmos lógicos, escaláveis e reutilizáveis.



Síntese da Parte 1

- Arrays organizam coleções de dados do mesmo tipo em índices numerados — de listas simples (1D) a estruturas em linhas e colunas (2D) e além (multidimensionais).

Arrays podem se aninhar — arrays dentro de arrays — para representar informação em camadas, como turmas, cardápios ou malas organizadas.

Por sua estrutura consistente, arrays são contêineres universais: previsíveis, reutilizáveis e essenciais para algoritmos escaláveis.



A seguir nesta aula

Parte 2 — Arrays Multidimensionais em Java.